TESTE FUNCIONAL

Black-box testing

Tópicos Especiais em Engenharia de Software

Sabrina Souto

Caso de Teste

O que é um **caso de teste**?

- Um conjunto de **valores de entrada**, **pré-condições** de execução, **resultados esperados** e **pós-condições** de execução, desenvolvido para cobrir condições de teste

- Um **bom caso de teste** é aquele que apresenta uma elevada probabilidade de revelar um bug ainda não descoberto

Cenário de Teste

- É uma história hipotética usada para ajudar as pessoas a solucionar um programa complexo, recriando ou visualizando um caminho a ser seguido

Cenário de Teste é a situação a ser testada!

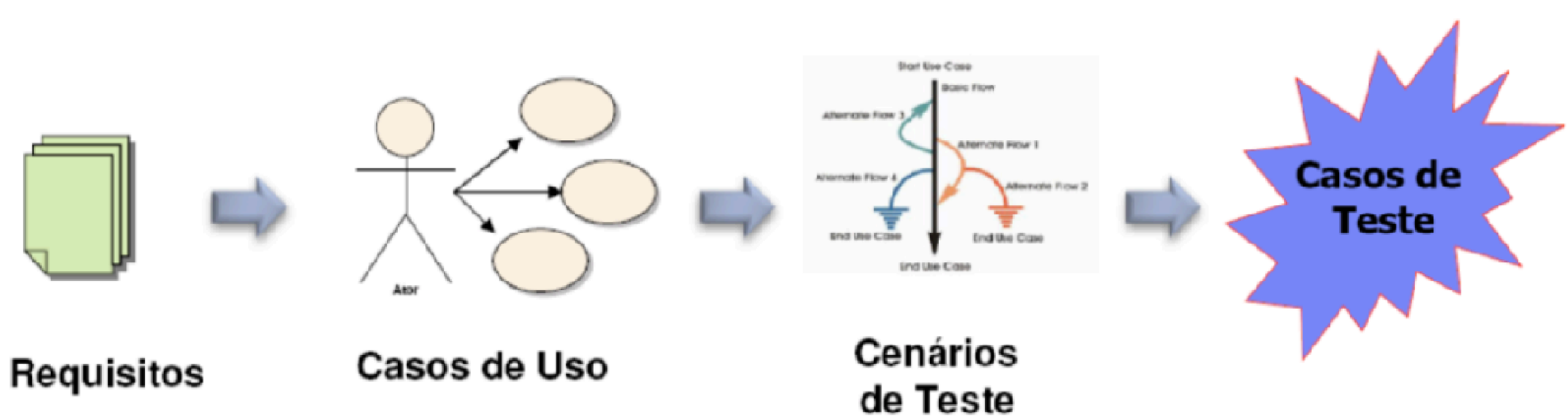
- **Exemplo de Cenário de Teste:**

1. Abrir a tela de cadastro de clientes
2. Clicar no botão "Novo Cliente"
3. Informar o CPF
4. Informar demais campos obrigatórios
5. Clicar em 'Cadastrar'

- **Exemplo de Casos de Teste:**
- CT 001: Validação de CPF
- CT 002: Preenchimento dos campos obrigatórios na tela
- CT 003: Data de Nascimento
- Inválida

Origem dos casos de teste —> Intuição

- Os casos de teste derivam dos requisitos do sistema



Composição de um caso de teste

- Um caso de teste é composto por:
  - Nome do caso de teste: “CT001”, por exemplo
  - Pré-condições
  - Descrição ou procedimentos
  - Resultado esperado
  - Massa de dados
  - Critérios especiais
  - Ambiente
  - Implementação
  - Iteração

Exemplo de caso de teste

Caso de Teste:	**CT 001 – Senha Inválida**
Pré-condições	Estar na tela de login do Administrador
Procedimento	1) O ator informa uma senha inválida e preenche um login válido 2) O ator seleciona a opção OK 3) O sistema verifica se os campos obrigatórios foram preenchidos 4) O sistema verifica se o login do usuário está cadastrado no sistema e se a senha é correta; 5) O sistema exibe a mensagem "Login/Senha inválidos"
Resultado esperado	Mensagem de erro do sistema
Dados de entrada	Um login inválido
Critérios especiais	Não se aplica
Ambiente	Windows XP; Internet Explorer 6.0 Service Pack 2; Servidor de aplicação Apache 2.0
Implementação	Manual
Iteração	1º Iteração

Como projetar casos de teste?

Teste funcional
Teste estrutural
Teste baseado em erros

Teste Funcional ou de Caixa-preta (*Black-box*)

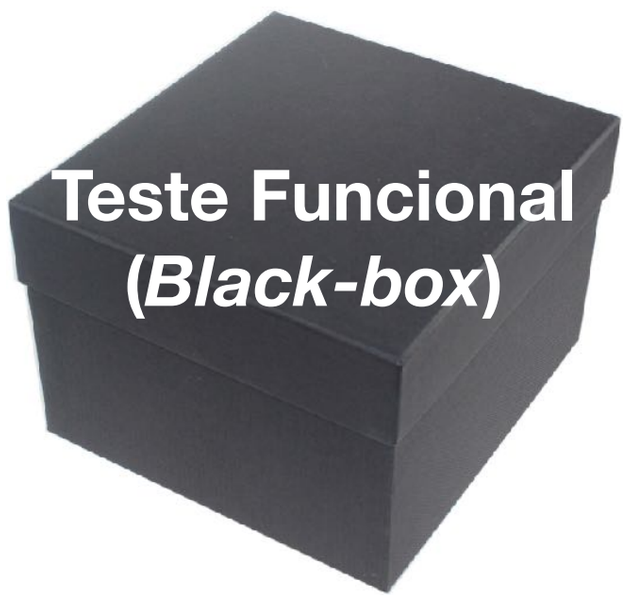
- O software a ser testado é abordado como se fosse uma caixa-preta, ou seja, tudo que tem dentro é ignorado

- O contrário do teste estrutural (*white-box*), onde o código do programa é a base para o teste.

Se o programa não é a base, então o que é?

Sua especificação!

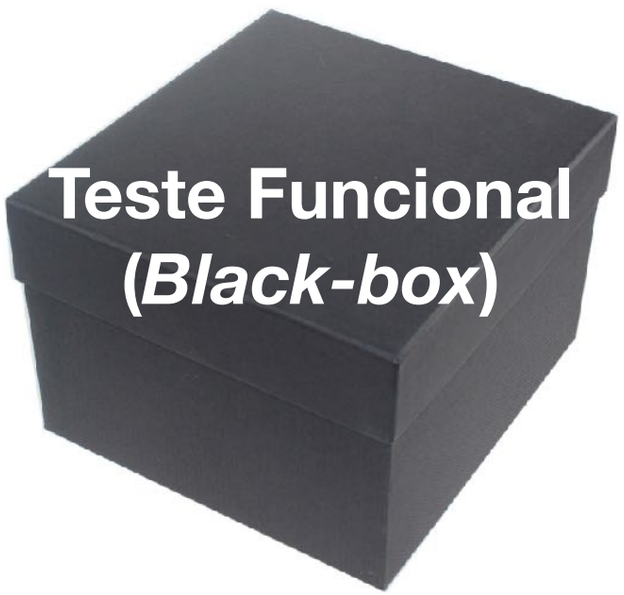
Tática de teste



- Teste baseado em especificação
- Teste cobre o máximo possível do comportamento *especificado*

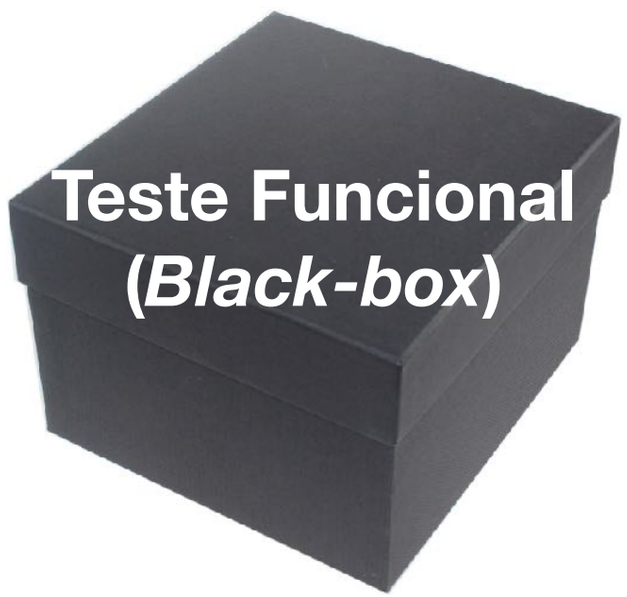
- Teste baseado no código
- Teste cobre o máximo possível do comportamento *implementado*

Por que Teste Funcional?



- Não é necessário o código do programa
- Projetar testes funcionais logo no inicio do projeto tem benefícios:
  - *revela problemas na especificação *prover uma explanação adicional da especificação *pode até servir como especificação, como em XP

Por que Teste Funcional?



- É eficaz em encontrar algumas classes de falhas que escapam às técnicas estruturais ou baseadas em falhas —> teste estrutural não foca no código que não está lá!

- Se aplica a todos os níveis de granularidade:

  - *teste de unidade *teste de integração *teste de sistema *teste de regressão

Desafio!

```
class Roots {
    // Solve ax² + bx + c = 0
    public roots(double a, double b, double c)
    { … }

    // Result: values for x
    double root_one, root_two;
}
```

- Quais valores para a, b, c devemos testar?

Assumindo a, b, c, como inteiros de 32 bits,
teríamos (2^32)^3 ≈ 10^28 entradas válidas
com 1.000.000.000.000 testes/segundo,
o que requer 2.5 bi anos.

Teste Exaustivo!!!

É impraticável executar todas as possibilidades!

Como escolher casos de teste?

- Alguns testes serão melhor que outros, no sentido de revelar falhas
- Não se pode saber antecipadamente quais casos de teste revelarão falhas
- No mínimo, podemos observar que executar o mesmo teste novamente tem menos chances de revelar uma falha do que executar um caso de teste diferente
- Podemos supor que um caso de teste muito diferente dos anteriores é mais interessante do que um que seja similar

Teste Aleatório/Randômico (*Random Testing*)

- Costuma ser um modo barato de produzir um grande número de casos de teste
- Pega todas as entradas uniformemente, ou seja, não prioriza uma entrada em relação à outra
- Se não existir um conhecimento prévio para priorizar mais um caso de teste que outro —> a abordagem randômica maximiza o ganho por maximizar o número de casos de teste que podem ser criados/executados
- Evita que o programador seja tendencioso
  - O programador pode cometer os mesmos erros no código e criar tendências ao testar (especialmente se são a mesma pessoa)

Por que não Teste Randômico?

- Defeitos não são distribuídos uniformemente —> o teste randômico pode deixar escapar bugs específicos em um grande espaço de entrada

- Assumindo que *Roots* aplica a equação quadrática e falha se b^2 - 4ac = 0 e a = 0

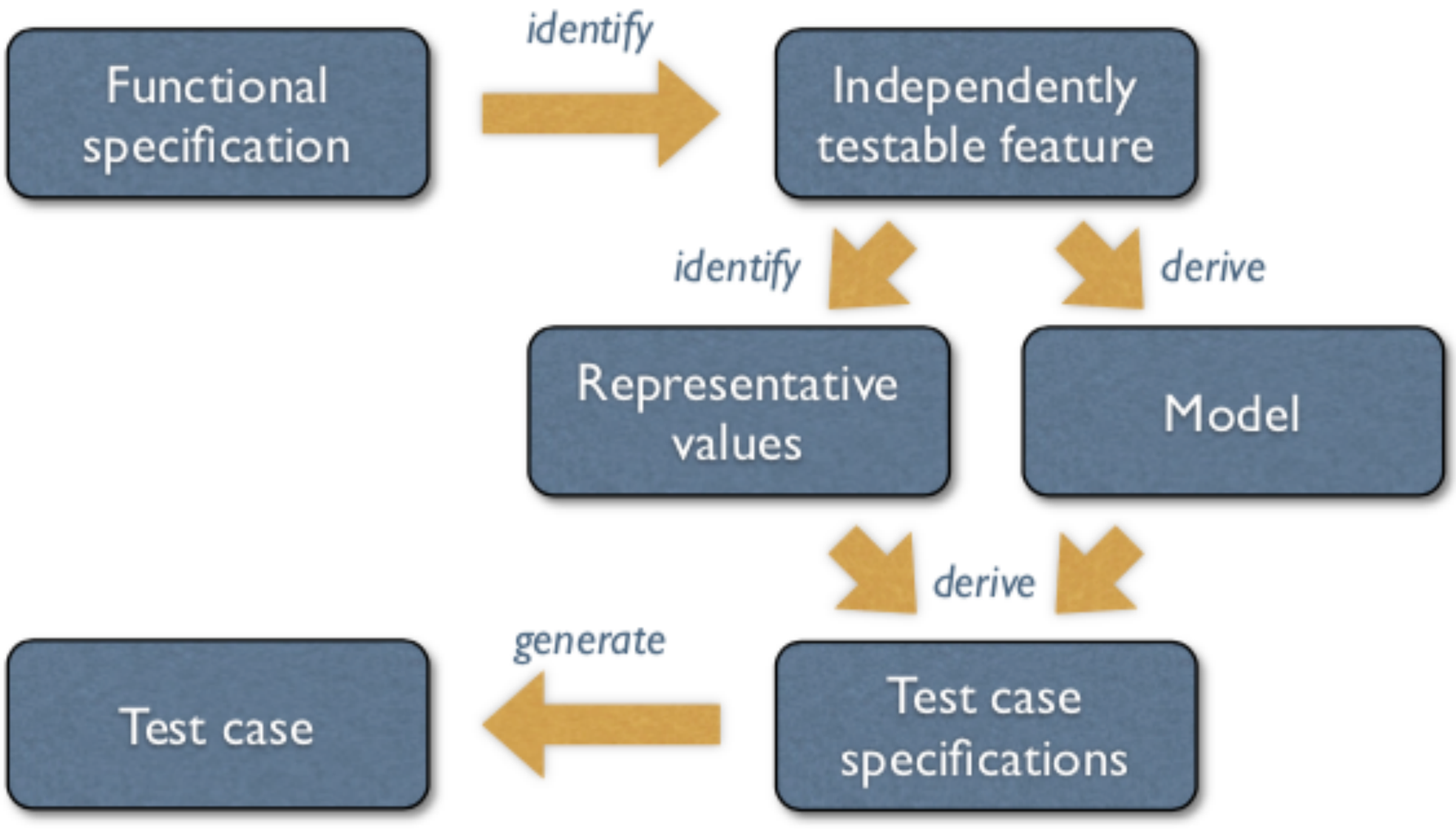
$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

- O teste randômico dificilmente escolheria a = 0 e b = 0

Teste Funcional Sistemático

- A derivação de casos de teste a partir de especificações funcionais é um processo analítico e complexo
- Força bruta raramente produz resultados aceitáveis
  - Os casos de teste são gerados sem algum critério particular
  - Determinar a adequação dos casos de teste é quase impossível
  - Baseia-se na capacitação do projetista de teste e é um processo difícil de monitorar e repetir
- Uma abordagem sistemática simplifica o processo, dividindo-o em etapas a partir de critérios para identificar conjuntos adequados de casos de teste, fornecendo meios de monitorar essa atividade

Teste Funcional Sistemático



Teste Funcional Sistemático: Identificar Funcionalidades Testáveis Independentemente

- Embora seja possível projetar casos de teste que exercitem diversas funcionalidades de uma vez
- Projetar casos de teste diferentes para funcionalidades diferentes
  - Pode simplificar o problema da geração de testes
  - Permitindo que cada funcionalidade seja examinada separadamente
  - Facilita a localização de bugs causadores das falhas reveladas

É recomendado elaborar casos de testes separados para cada funcionalidade

Teste Funcional Sistemático: Identificar Funcionalidades Testáveis Independentemente

- Etapa preliminar do teste funcional —> particionar a especificação em funcionalidades que possam ser testadas separadamente (*Independently Testable Features - ITF*)
- Uma ITF não necessariamente corresponde a unidades ou subsistemas
- ITFs são descritas identificando todas as entradas que formam seus ambientes de execução

Teste Funcional Sistemático: Funcionalidades Testáveis Independentemente

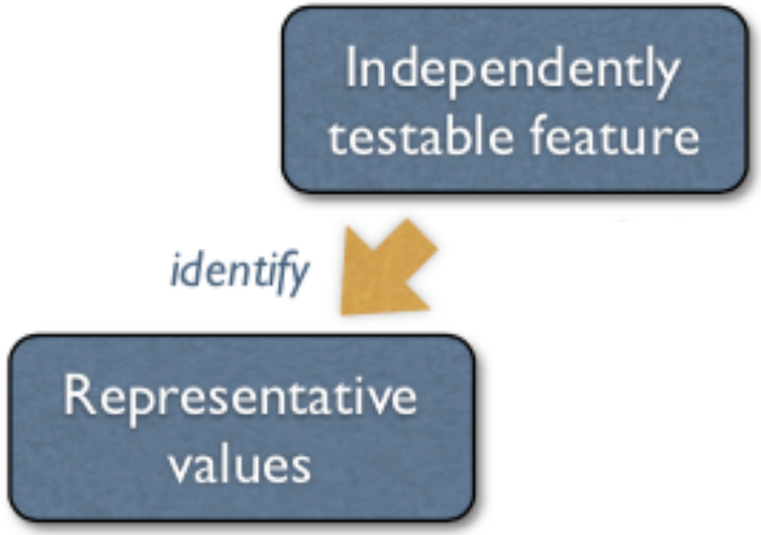
```
class Roots {
    // Solve ax² + bx + c = 0
    public roots(double a, double b, double c)
    { … }

    // Result: values for x
    double root_one, root_two;
}
```

- Quais são as funcionalidades testáveis independentemente?

Teste Funcional Sistemático: Funcionalidades Testáveis Independentemente

- Considere uma calculadora
- Quais as unidades testáveis independentemente?

Valores Representativos

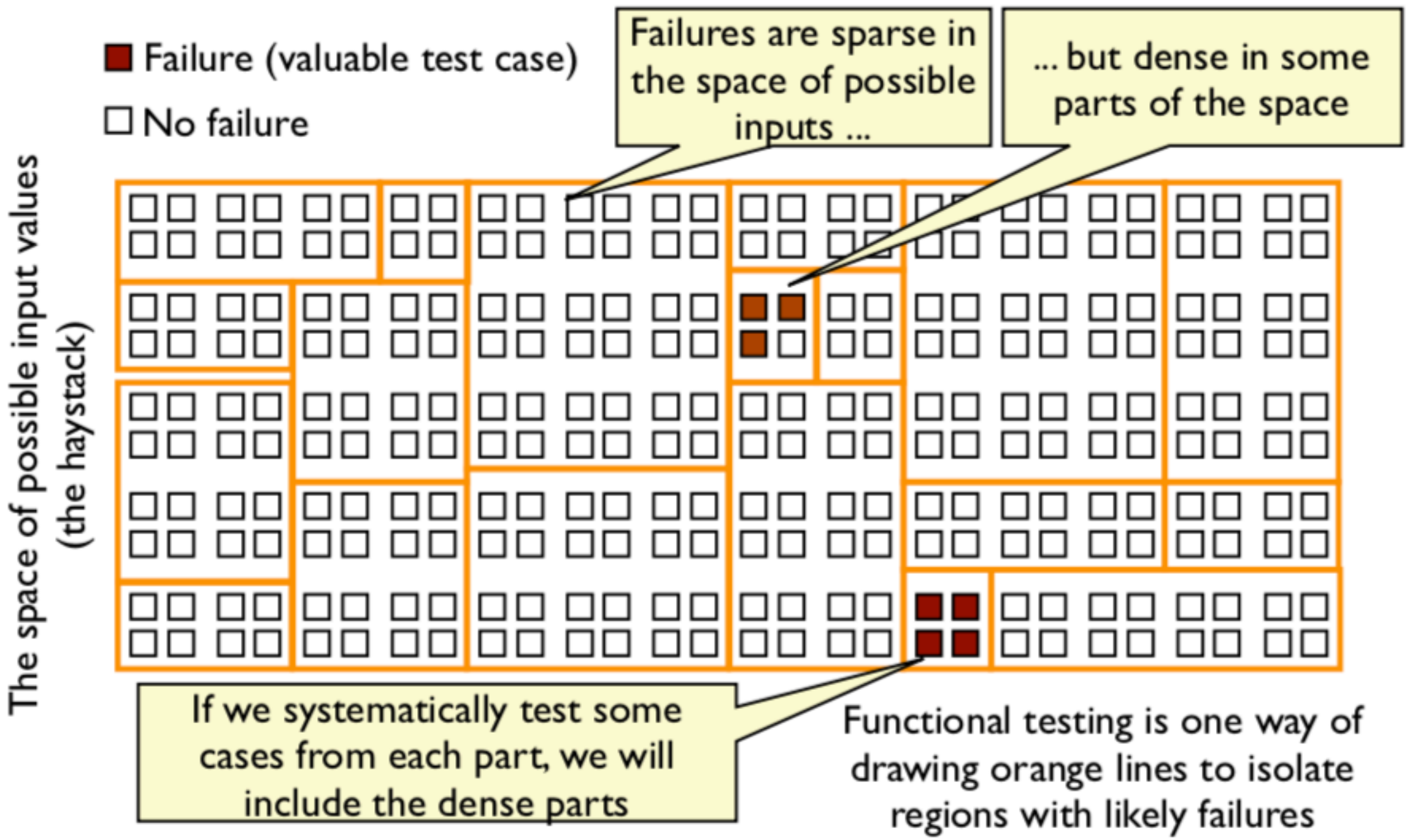


- Tenta selecionar entradas relevantes —> escolhendo classes de equivalência representativas no sentido de revelar falhas
- TESTE DE PARTIÇÃO —> separa o espaço de entrada em classes de equivalência cuja união é o espaço inteiro, mas as classes não precisam ser disjuntas

Uma agulha num palheiro

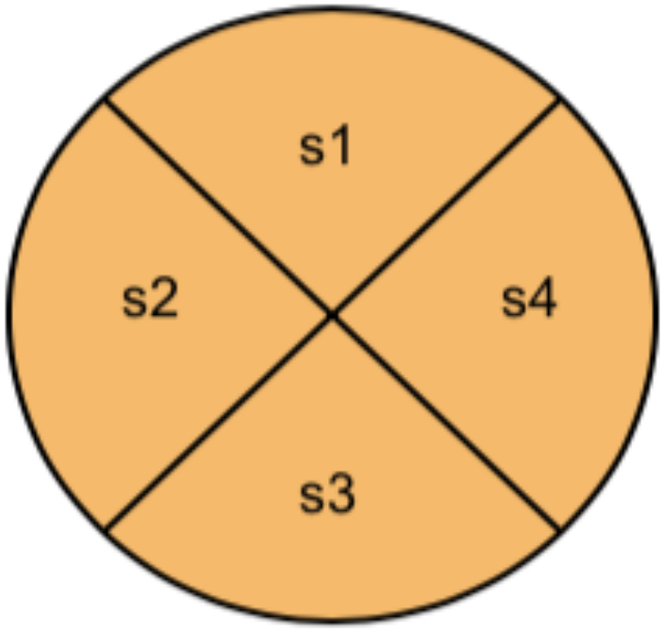
- Para achar a agulha, procure sistematicamente!
- Precisamos encontrar o que torna a agulha especial/relevante/diferente

Teste de Partição Sistemático



Partição por Equivalência

- As entradas são divididas em partições de equivalência —> o comportamento do sistema será o mesmo para qualquer entrada escolhida em uma mesma partição
- Não é preciso testar todas as entradas possíveis, basta testar uma opção de cada partição
- Reduz a quantidade de casos de teste a um nível controlável mantendo uma cobertura razoável



Partição por Equivalência

- Como escolher classes de equivalência?
  - Examinar as condições de entrada de acordo com a especificação —> valores válidos e inválidos

Input condition	Equivalence classes
range	one valid, two invalid (larger and smaller)
specific value	one valid, two invalid (larger and smaller)
member of a set	one valid, one invalid
boolean	one valid, one invalid

Partição por Equivalência

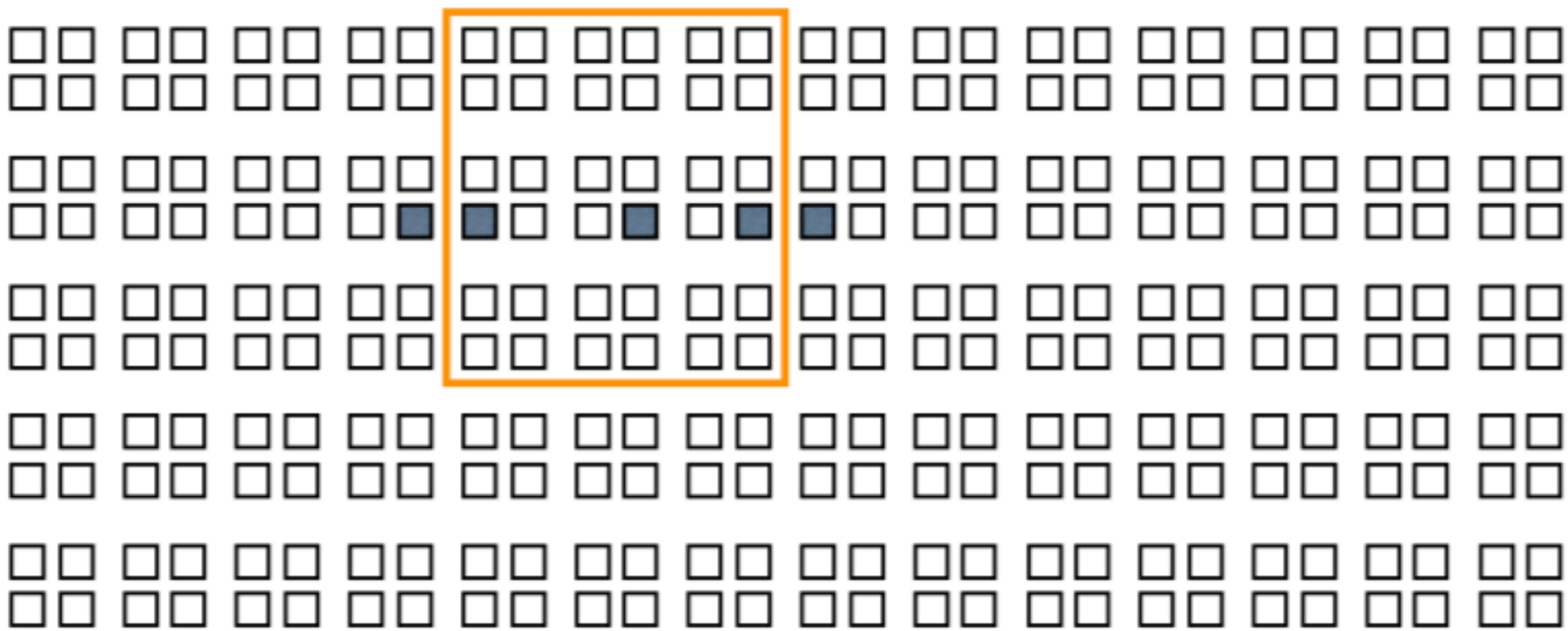
- Exemplo: em um sistema de gestão de contratos, a idade dos clientes varia entre 18 e 68 anos

Entrada	Valores Permitido	Classes	Casos de teste
Idade	Idade entre 18 e 65	18 a 68	Idade = 20
		< 18	Idade = 10
		> 68	Idade = 80

Análise do Valor Limite

- Como escolher classes de equivalências representativas?
- Nos limites de uma partição é onde existe maior probabilidade de revelar falhas
- Os valores limites de uma partição são seu máximo e seu mínimo, que podem ser válidos ou inválidos —> testes podem ser projetados para cobrir valores válidos e inválidos

Análise do Valor Limite



- Testar os valores nos limites inferiores (válidos e inválidos), nos limites superiores (válidos e inválidos) e no centro.

Análise do Valor Limite

- Considere as seguintes regras organizacionais, sobre a contratação baseada em idade:

  0-16 Não Contrata

  16-18 Contrata meio-expediente

  18-55 Contrata tempo integral

  55-99 Não Contrata

Análise do Valor Limite

- Uma regra correta seria:

```
Se (idadeCandidato >= 0 && idadeCandidato <= 15)
estadoContratação = "Não";
Se (idadeCandidato >= 16 && idadeCandidato <= 17)
estadoContratação = "Meio";
Se (idadeCandidato >= 18 && idadeCandidato <= 54)
estadoContratação = "Integral";
Se (idadeCandidato >= 55 && idadeCandidato <= 99)
estadoContratação = "Não";
```

- Conjuntos de valores interessantes para serem testados: {-1,0,1}, {15,16,17},{17,18,19},{54,55,56} e {98,99,100}

- Dependendo das pré-condições, valores como {-42, 1001, FRED, %$#@} também devem ser testados!

Estudar

- Capítulo 10 - Livro texto

Referências

- Livro texto: Pezze, M., Young, M. Teste e Análise de Software Processos, Princípios e Técnicas, editora Bookman, 2008.

- Testing and Debugging Advanced Course - Summer September 2010. Saarland University. Gordon Fraser and Andreas Zeller. URL: https://www.st.cs.uni-saarland.de/edu/testingdebugging10/